BANNARI AMMAN INSTITUTE OF TECHNOLOGY
An Autonomous Institution Affiliated to Anna University - Chennai • Approved by AICTE • Accredited by NAAC with 'A+' Grade
SATHYAMANGALAM - 638401 ERODE DISTRICT TAMILNADU INDIA
Ph: 04295-226000/221289 Fax: 04295-226666 Email: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

S5 PROJECT WORK I | SECOND REVIEW

AUTOMATED SOFTWARE TESTING AND BUG ANALYSIS

(SentinelMesh)

S5-CSE-2026-195

INTERNAL - OWN PROJECT

Hariesh P (7376241CS195) - Dept. of CSE
Midhum M (7376242AL152) - Dept. of AIML

Ms. Ramya R (CS1906)
Assistant Professor
Dept. of Computer Science and Engineering

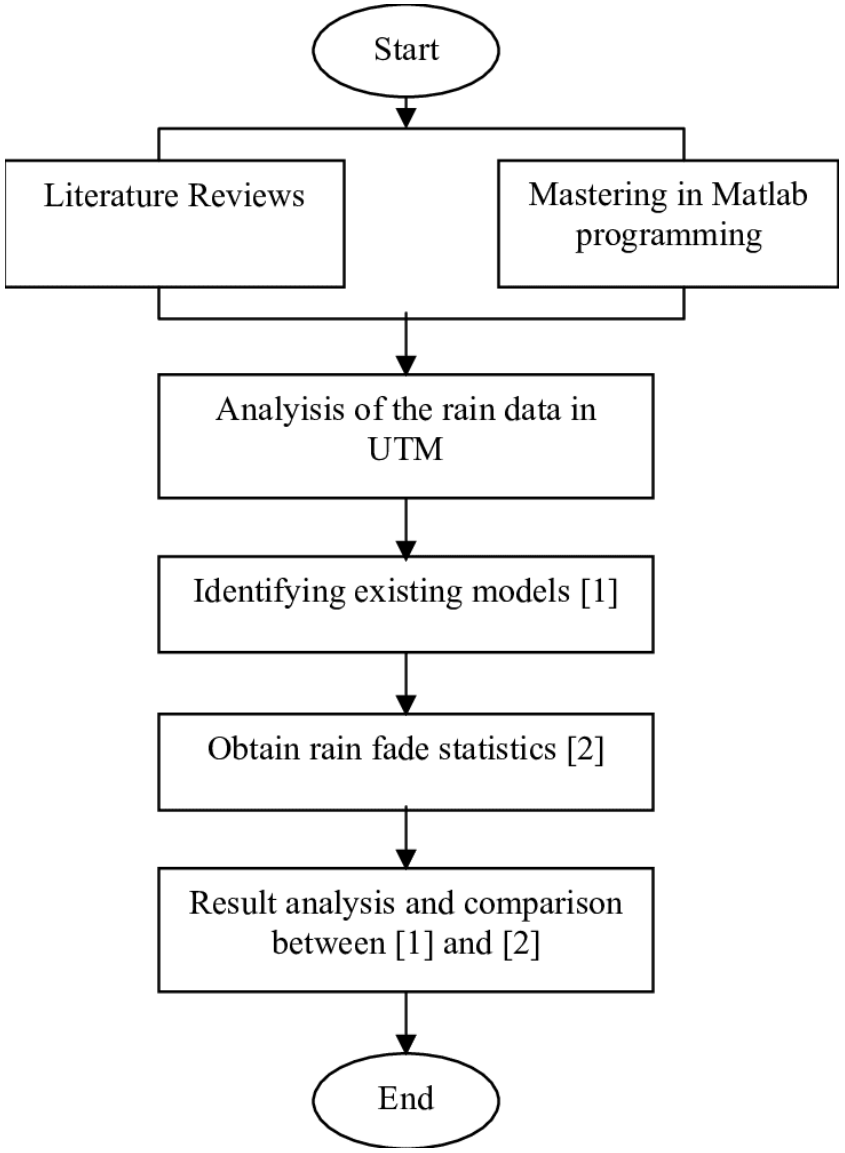
- Identified limitations in manual software testing and defect triage workflows.
- Reviewed static analysis, mutation testing, and multi-agent coordination research.

- **Phase 2 - Architecture & Agent Design (Weeks 2-3)**
- Designed 9-agent coordination graph: Orchestrator, Static Analysis, Test Generation,
- Test Execution, Bug Classification, Root Cause, Auto-Fix, Critic, Report agents.

- **Phase 3 - Implementation (Weeks 3-5)**
- Implemented AST-based analysis, test synthesis, sandbox execution, and reporting.
- Integrated FastAPI backend, SQLite/SQLAlchemy, and React mission-control dashboard.

- **Phase 4 - Validation & Documentation (Weeks 6-7)**
- Validated: 37/37 tests passing, 85% platform code coverage.
- Prepared full academic project report and Second Review presentation.

- Core agent logic, AST parsing, sandboxed test execution, and backend services.
- **FastAPI + Uvicorn**
- RESTful API gateway, WebSocket communication, and health-monitoring endpoints.
- **Python AST Module**
- Structural code parsing, cyclomatic-complexity scoring, security-smell detection.
- **SQLite + SQLAlchemy ORM**
- Persistent storage of analysis results, test records, patch history, agent state.
- **React 18 (Vite)**
- Mission-control dashboard: real-time telemetry, risk heat-map, report viewer.
- **pytest + coverage.py**
- 37/37 automated tests passing; 85% platform code coverage measured.
- **Subprocess Sandbox**
- Isolated test execution with 20-second timeout and archive-size controls.
- **Git + GitHub**
- Version control - github.com/HARIESHPERIASAMY/S5_Mini_Project



FastAPI gateway validates and routes to Orchestrator Agent

Static Analysis Agent >> AST parsing, risk scoring, security-smell detection

Test Generation Agent >> unit, boundary, edge-case & mutation test cases

Test Execution Agent >> sandbox run, coverage collection

Bug Classification Agent >> severity tagging, duplicate detection


Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
Research
Content
Design
Uploading
Developing
Finalizing
Marketing

- Manages execution graph, agent retries, and lifecycle state transitions.

- **2. Static Analysis Agent**
- AST parsing, cyclomatic complexity, security-smell detection, risk scores per file.
- **3. Test Generation Agent**
- Prioritised unit, boundary, edge-case, and property-based pytest case generation.
- **4. Test Execution Agent**
- Runs tests in isolated subprocess sandbox; tracks coverage and flaky-test flags.
- **5. Bug Classification Agent**
- Labels defect type, severity level; detects duplicate findings.
- **6. Root Cause Agent**
- Maps stack traces to source locations; generates causal explanations.
- **7. Auto-Fix Agent**
- Proposes unified-diff patches; verifies by re-running tests in sandbox.
- **8. Critic / Reviewer Agent**
- Challenges findings via debate; computes consensus quality score.
- **9. Report Agent**
- Builds executive & technical reports; streams telemetry to React dashboard.

- Nine specialised agents divide testing tasks - no single point of failure.
- AST-based static analysis provides deep source-level risk signals.
- Subprocess sandbox limits execution time (20 s) and extraction size.
- Patch proposals are re-tested in sandbox before being marked verified.
- Structured JSON reports and risk heat-map improve review visibility.
- Offline deterministic fallback - works without a cloud LLM provider.
- 37/37 tests passing; 85% platform code coverage validated.

LIMITATIONS

- Static and test-generation rules are currently Python-focused.
- Rule-based detectors may miss context-dependent or semantic defects.
- LLM-assisted features depend on external API quotas (configurable).
- Sandbox 20-second timeout may be insufficient for very large codebases.
- Auto-generated tests do not guarantee complete path/branch coverage.
- Multi-language support (Java, C++) is planned future work.

- software quality lifecycle - from static analysis to sandbox-verified auto-fix.

- Nine dedicated agents collaborate via an event-driven bus without tight coupling,
- enabling independent extensibility and testing of each component.

- FastAPI backend, SQLite persistence, sandboxed execution, and a React dashboard
- provide a production-grade foundation for local-first quality engineering.

- Validated result: 37/37 automated tests passing; 85% platform code coverage.

- Future directions: multi-language analysis (Java, C++, Go), cloud deployment,
- LLM-augmented patch generation, and cross-repository trend analytics.

- **Outcome:**
- A fully traceable workflow from source-code submission to structured bug evidence,
- root-cause explanation, sandbox-verified patches, and exportable executive report.

- Background should be White in all slides
- Font
  - Cambria
  - Font Size & Colors
    - Headings – **Blue – UPPER CASE – Bold** (Font Size 24)
    - Subheadings **Maroon / Violet / Green - Sentence Case- Bold** (Font Size 24)
    - Black for Contents (Font Size 20)

- 1. Designed overall system architecture and 9-agent coordination graph.
- 2. Implemented Orchestrator, Static Analysis, Bug Classification & Root Cause agents.
- 3. Developed FastAPI backend: API gateway, WebSocket endpoints, health checks.
- 4. Built SQLAlchemy ORM models, database schema, and migration scripts.
- 5. Wrote 37 automated unit and integration tests; achieved 85% code coverage.
- 6. Authored full academic project report and Second Review documentation.
- 7. Managed Git repository and maintained clean commit history on GitHub.

- 1. Implemented Test Generation, Test Execution, and Auto-Fix agents.
- 2. Designed and integrated the subprocess sandbox with timeout & size controls.
- 3. Developed Critic / Reviewer Agent for consensus scoring and debate logic.
- 4. Built Report Agent and executive / technical report generation pipeline.
- 5. Developed React 18 (Vite) mission-control dashboard with real-time telemetry.
- 6. Integrated frontend with backend WebSocket streams and risk heat-map.
- 7. Assisted in validation testing, debugging, and Second Review preparation.

- **Client Layer**
- React 18 dashboard: repository upload, live agent log, risk heat-map, report download.
- **API Gateway**
- FastAPI + Uvicorn: REST endpoints, WebSocket streams, CORS, request validation.
- **Agent Orchestration Layer**
- Event-driven bus coordinates 9 agents with retry, timeout, lifecycle management.
- **Analysis & Execution Layer**
- Python AST engine, pytest runner, unified-diff patch applier, subprocess sandbox.
- **Persistence Layer**
- SQLite + SQLAlchemy ORM: analysis sessions, test records, patches, agent logs.

	Manuscript in preparation: SentinelMesh - Multi-Agent Automated Software Testing using AST Analysis and Sandboxed Execution	
	No conference/journal/patent submitted as of Second Review date.	

	Turnitin / iThenticate	
	Grammarly Plagiarism Checker (secondary)	

- [2] Python Software Foundation, ast - Abstract Syntax Trees, Python 3.11 Docs. https://docs.python.org/3/library/ast.html
- [3] T. Ramirez, FastAPI Documentation, Tiangolo, 2024. https://fastapi.tiangolo.com/
- [4] Pytest Development Team, pytest Documentation, 2024. https://docs.pytest.org/
- [5] SQLAlchemy Authors, SQLAlchemy Documentation, 2024. https://docs.sqlalchemy.org/
- [6] React Authors, React 18 Documentation, Meta Open Source, 2024. https://react.dev/
- [7] M. Wooldridge and N. Jennings, Intelligent Agents: Theory and Practice, Knowledge Engineering Review, vol.10, no.2, pp.115-152, 1995.
- [8] A. Zeller, Why Programs Fail: A Guide to Systematic Debugging, 2nd ed., Morgan Kaufmann, 2009.
- [9] P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed., Cambridge University Press, 2016.
- [10] S. Yoo and M. Harman, Regression Testing Minimisation, Selection and Prioritisation: A Survey, STVR, vol.22, no.2, pp.67-120, 2012.
- [11] G. Fraser and A. Arcuri, EvoSuite: Automatic Test Suite Generation for Object-Oriented Software, in Proc. FSE, 2011, pp.416-419.
- [12] R. Feldt et al., Finding test data in existing code, in Proc. ICSME, 2013, pp.1-10.
- [13] M. Harman and B. Jones, Search-based software engineering, Information & Software Technology, vol.43, no.14, pp.833-839, 2001.